

DOCUMENTACIÓN TÉCNICA

Recicley Watermark

Marcado de agua por lotes para archivos PDF — Manual de uso y explicación del código fuente

Archivo fuente documentado: GUI_Watermark.py

Framework de interfaz: PyQt6

Plataforma: Windows 10/11 (64 bits)

Fecha del documento: 18 de julio de 2026

Índice

1. Descripción general
2. Instalación y requisitos
3. Guía de uso de la aplicación
4. Arquitectura general del código
5. Módulos importados y constantes globales
6. La clase `DragDropWindow` — construcción de la interfaz
7. Gestión del estado: lista de PDFs, marca de agua y destino
8. Diálogos de selección de archivos
9. Arrastrar y soltar (Drag & Drop)
10. `processWatermark` — el orquestador del procesamiento por lotes
11. `addWaterMarkToPDF` — el motor que aplica la marca de agua
12. Punto de entrada `main()`
13. Flujo de datos completo (paso a paso)
14. Empaquetado a `.exe` (PyInstaller)
15. Solución de problemas

1. Descripción general

Recicley Watermark es una aplicación de escritorio para Windows, construida en Python con la librería gráfica **PyQt6**, que permite aplicar una imagen de marca de agua (con transparencia) sobre uno o varios archivos PDF de forma idéntica. El usuario carga los PDF (arrastrándolos, seleccionándolos manualmente o apuntando a una carpeta completa), carga una imagen de marca de agua, ajusta la opacidad deseada y, con un solo clic, la aplicación genera copias marcadas de todos los documentos.

Internamente, cada página del PDF se convierte a una imagen de alta resolución (usando **Poppler** a través de la librería `pdf2image`), se combina con la marca de agua usando composición alfa de **Pillow** (`PIL`), y las páginas resultantes se reensamblan en un nuevo archivo PDF con **PyPDF2**.

2. Instalación y requisitos

Uso como ejecutable (recomendado)

No requiere Python ni ninguna instalación. Basta con descomprimir la carpeta `GUI_Watermark/` (que contiene `GUI_Watermark.exe` y la subcarpeta `_internal/`) y ejecutar el `.exe`. La carpeta `_internal` debe permanecer junto al ejecutable: ahí viven las DLL de PyQt6, los binarios de Poppler, el ícono de la aplicación y demás dependencias empaquetadas.

Importante: nunca copies solo el `.exe` a otra máquina sin la carpeta `_internal`; la aplicación no podrá iniciar porque le faltarán sus dependencias.

Ejecución desde el código fuente

Requisito	Uso
Python 3.13	Intérprete del lenguaje.
PyQt6	Interfaz gráfica (ventana, botones, slider, diálogos, drag & drop).
pdf2image	Convierte páginas de un PDF en imágenes, delegando en Poppler.
PyPDF2	Lee/escribe archivos PDF página por página.
Pillow (PIL)	Manipulación de imágenes: redimensionar, canal alfa, composición.
poppler-0.68.0/bin	Binarios nativos de Poppler que <code>pdf2image</code> invoca para renderizar.

3. Guía de uso de la aplicación

- 1 Cargar PDF(s):** arrastra uno o varios archivos `.pdf`, o una carpeta completa (se agregan recursivamente todos los PDF que contenga), a la ventana. También puedes usar el botón **Archivos** (selector múltiple) o **Carpeta** (selector de carpeta). El botón **Limpiar** vacía la lista.
- 2 Cargar la marca de agua:** arrastra una imagen `.png` / `.jpg` / `.jpeg` o usa el botón **Elegir imagen**. Se recomienda un PNG con canal alfa (fondo transparente) para un resultado más limpio.
- 3 Elegir destino (opcional):** por defecto los archivos resultantes van a la carpeta *Descargas* del usuario. El botón **Elegir destino** permite cambiarla.
- 4 Ajustar opacidad:** el control deslizante define qué tan visible es la marca de agua, de 0 (invisible) a 255 (totalmente opaca).

5

Procesar: al pulsar **Agregar Marca de Agua**, se recorre cada PDF de la lista, mostrando una barra de progreso cancelable. Al terminar se muestra un resumen y se abre automáticamente la carpeta de salida en el explorador de Windows.

4. Arquitectura general del código

Todo el proyecto vive en un único archivo, `GUI_Watermark.py`, organizado en tres bloques conceptuales:

1. **Configuración global:** imports, constantes de calidad/resolución y la función auxiliar `get_base_path()`, usada para localizar recursos empaquetados (el ícono y Poppler) tanto en modo script como en modo ejecutable.
2. **La interfaz gráfica y su lógica:** la clase `DragDropWindow(QMainWindow)`, que concentra tanto la construcción visual de la ventana como todos los manejadores de eventos (clics, arrastre, progreso).
3. **El punto de entrada:** la función `main()`, que crea la aplicación Qt y muestra la ventana.

No existe separación en múltiples archivos ni patrón MVC formal: al ser una utilidad de escritorio pequeña, la clase `DragDropWindow` actúa a la vez como *vista* (widgets) y *controlador* (lógica de negocio), delegando el procesamiento pesado de imágenes a librerías externas (Pillow, pdf2image, PyPDF2).

5. Módulos importados y constantes globales

```
import sys
import os
import io
from datetime import datetime
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QLabel, QWidget, QPushButton, QMessageBox,
    QSlider, QFileDialog, QProgressDialog
)
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt

from pdf2image import convert_from_bytes
from PyPDF2 import PdfWriter, PdfReader
from PIL import Image
```

Módulo	Para qué se usa en este proyecto
<code>sys</code>	Acceso a <code>sys.argv</code> (arranque de la app Qt), <code>sys.exit()</code> y detección de si el programa corre "congelado" (empaquetado como <code>.exe</code>) vía <code>sys.frozen</code> / <code>sys._MEIPASS</code> .

<code>os</code>	Rutas de archivos y carpetas: unir rutas, listar/recorrer directorios (<code>os.walk</code>), obtener la carpeta de usuario, crear carpetas, abrir el explorador (<code>os.startfile</code>).
<code>io</code>	Buffers de memoria (<code>io.BytesIO</code>) para generar PDFs e imágenes intermedias sin escribir archivos temporales en disco.
<code>datetime</code>	Genera la marca de tiempo usada en el nombre de la subcarpeta de salida cuando se procesan varios PDFs a la vez.
<code>PyQt6.QtWidgets</code>	Todos los widgets de la interfaz: ventana principal, etiquetas, botones, slider, cuadros de diálogo de archivo/carpeta y la barra de progreso.
<code>PyQt6.QtGui.QIcon</code>	Carga el ícono de la ventana (<code>rec.jpg</code>).
<code>PyQt6.QtCore.Qt</code>	Constantes de Qt: orientación del slider, modalidad de la ventana de progreso.
<code>pdf2image.convert_from_bytes</code>	Convierte los bytes de un PDF (una página) en una imagen de Pillow, invocando internamente los binarios de Poppler.
<code>PyPDF2.PdfWriter</code> / <code>PdfReader</code>	Lectura del PDF original página por página y escritura del PDF final con las páginas ya marcadas.
<code>PIL.Image</code>	Redimensionar la marca de agua, ajustar su canal alfa según la opacidad elegida y componerla sobre cada página renderizada.

```

RENDER_DPI = 200
PDF_SAVE_QUALITY = 95
IMAGE_EXTENSIONS = ( '.png', '.jpg', '.jpeg' )

```

`RENDER_DPI` controla la resolución a la que Poppler renderiza cada página del PDF antes de aplicar la marca de agua: a mayor DPI, más nítido el resultado pero más lento y pesado el proceso. `PDF_SAVE_QUALITY` es la calidad JPEG usada por Pillow al reinsertar cada página como PDF. `IMAGE_EXTENSIONS` centraliza las extensiones aceptadas como imagen de marca de agua, para no repetirlas en cada validación.

función`get_base_path()`

```
def get_base_path():
    if getattr(sys, 'frozen', False):
        return getattr(sys, '_MEIPASS', os.path.dirname(sys.executable))
    return os.path.dirname(os.path.abspath(__file__))
```

Devuelve la carpeta desde la que deben leerse los recursos empaquetados (`rec.jpg` y `poppler-0.68.0/`). Cuando el programa corre como script normal, es la carpeta donde está `GUI_Watermark.py`. Cuando corre como `.exe` generado por PyInstaller (`sys.frozen` es `True`), usa `sys._MEIPASS`: la carpeta interna donde PyInstaller coloca los archivos de datos empaquetados (la carpeta `_internal` junto al ejecutable). Esta función es la que hace posible que el mismo código funcione igual en desarrollo y una vez compilado.

6. La clase `DragDropWindow` — construcción de la interfaz

clase

`DragDropWindow(QMainWindow)` hereda de `QMainWindow` y representa la única ventana de la aplicación. Su método `__init__` hace tres cosas: configura la ventana, crea todos los widgets con su posición/estilo, y conecta cada control con el método que debe ejecutarse al interactuar con él.

6.1 Configuración de la ventana

```
self.setAcceptDrops(True)
self.setWindowTitle("Recicley Watermark")
icon_path = os.path.join(get_base_path(), "rec.jpg")
self.setWindowIcon(QIcon(icon_path))
self.setMinimumSize(500, 520)
self.setMaximumSize(500, 520)
```

`setAcceptDrops(True)` habilita que la ventana reciba eventos de arrastrar y soltar (necesarios para `dragEnterEvent` / `dropEvent`, ver sección 9). El tamaño se fija en 500×520 píxeles (mínimo = máximo), por lo que la ventana no es redimensionable; todos los widgets se posicionan con coordenadas absolutas (`setGeometry(x, y, ancho, alto)`) calculadas para encajar en ese espacio fijo.

6.2 Widgets y su rol

Widget	Tipo	Rol
<code>title</code>	<code>QLabel</code>	Título decorativo "Protector de archivos".
<code>label_pdf</code>	<code>QLabel</code>	Muestra cuántos/qué PDF están cargados (actualizado por <code>update_pdf_label()</code>).

<code>button_select_files</code>	<code>QPushButton</code>	Abre un diálogo para elegir uno o varios PDF manualmente.
<code>button_select_folder</code>	<code>QPushButton</code>	Abre un diálogo para elegir una carpeta; se agregan todos los PDF que contenga (recursivo).
<code>button_clear_files</code>	<code>QPushButton</code>	Vacía la lista de PDF cargados.
<code>label_watermark</code>	<code>QLabel</code>	Muestra el nombre de la imagen de marca de agua cargada.
<code>button_select_watermark</code>	<code>QPushButton</code>	Abre un diálogo para elegir la imagen de marca de agua.
<code>label_output</code>	<code>QLabel</code>	Muestra la carpeta de destino elegida, o el texto por defecto si no se ha elegido ninguna.
<code>button_select_output</code>	<code>QPushButton</code>	Abre un diálogo para elegir la carpeta de destino.
<code>opacity_label</code> / <code>opacity_slider</code>	<code>QLabel</code> / <code>QSlider</code>	Controlan y muestran la opacidad (0–255) que se aplicará a la marca de agua.
<code>button</code>	<code>QPushButton</code>	Botón principal "Agregar Marca de Agua"; dispara <code>processWatermark()</code> .

Cada botón se conecta a su manejador con la señal `clicked.connect(...)` de Qt, y el slider con `valueChanged.connect(self.updateOpacity)`, siguiendo el patrón estándar de señales/slots de PyQt.

método `updateOpacity(self)`

```
def updateOpacity(self):
    self.opacity = self.opacity_slider.value()
    self.opacity_label.setText(f"Opacidad: {self.opacity}")
```

Se ejecuta cada vez que el usuario mueve el slider. Simplemente refleja el valor numérico actual en la etiqueta de texto, para dar retroalimentación inmediata.

7. Gestión del estado: lista de PDFs, marca de agua y destino

El estado de la aplicación vive en tres atributos de instancia, inicializados en `__init__`:

- `self.pdf_paths`: lista de rutas absolutas a los PDF cargados (puede tener 0, 1 o N elementos).
- `self.watermark_path`: ruta de la imagen de marca de agua (una sola, aplicada a todos los PDF).
- `self.output_dir`: carpeta de destino elegida por el usuario, o cadena vacía si se usará la carpeta por defecto (Descargas).

método `add_pdf(self, path)`

```
def add_pdf(self, path):
    if path not in self.pdf_paths:
        self.pdf_paths.append(path)
```

Agrega una ruta a `pdf_paths` evitando duplicados (por ejemplo, si el usuario arrastra dos veces el mismo archivo, o si aparece tanto al arrastrar un archivo suelto como dentro de una carpeta arrastrada después).

método `add_pdfs_from_folder(self, folder)`

```
def add_pdfs_from_folder(self, folder):
    found = []
    for root, _dirs, files in os.walk(folder):
        for f in files:
            if f.lower().endswith('.pdf'):
                found.append(os.path.join(root, f))
    for f in sorted(found):
        self.add_pdf(f)
```

`os.walk(folder)` recorre la carpeta indicada y **todas sus subcarpetas** de forma recursiva. Cada archivo cuyo nombre termine en `.pdf` (sin distinguir mayúsculas/minúsculas) se agrega a una lista temporal, que luego se ordena alfabéticamente y se añade una a una vía `add_pdf` (reutilizando así su protección contra duplicados). Esta es la función que implementa "cargar todos los PDF de una ruta" pedido por el usuario: tanto el botón **Carpeta** como arrastrar una carpeta a la ventana llaman a este mismo método.

método `update_pdf_label(self)`

```
def update_pdf_label(self):
    n = len(self.pdf_paths)
    if n == 0:
        self.label_pdf.setText("Arrastra PDF(s) o una carpeta aquí")
    elif n == 1:
        self.label_pdf.setText(f"Archivo PDF: {os.path.basename(self.pdf_paths[0])}")
    else:
        self.label_pdf.setText(f"{n} archivos PDF seleccionados")
    ...
```

Sincroniza el texto (y el *tooltip*, que lista todos los nombres) de `label_pdf` con el contenido real de `self.pdf_paths`. Se llama después de cualquier operación que modifique la lista: arrastrar archivos, elegir archivos/carpeta por diálogo, o limpiar.

método

`clearFiles(self)`

```
def clearFiles(self):  
    self.pdf_paths = []  
    self.update_pdf_label()
```

Vacía la lista de PDF y refresca la etiqueta. Conectado al botón **Limpiar**.

8. Diálogos de selección de archivos

Estos cuatro métodos son manejadores de clic que abren los cuadros de diálogo nativos de Windows (`QFileDialog`) como alternativa a arrastrar y soltar.

Método	Diálogo de Qt usado	Efecto
<code>selectFileDialog</code>	<code>QFileDialog.getOpenFileNames</code> (filtro <code>*.pdf</code>)	Permite elegir varios PDF a la vez; cada uno se agrega con <code>add_pdf</code> .
<code>selectFolderDialog</code>	<code>QFileDialog.getExistingDirectory</code>	Elige una carpeta y delega en <code>add_pdfs_from_folder</code> para el escaneo recursivo.
<code>selectWatermarkDialog</code>	<code>QFileDialog.getOpenFileName</code> (filtro imágenes)	Reemplaza <code>self.watermark_path</code> por la imagen elegida.
<code>selectOutputDialog</code>	<code>QFileDialog.getExistingDirectory</code>	Fija <code>self.output_dir</code> con la carpeta elegida y actualiza <code>label_output</code> .

En los cuatro casos, tras modificar el estado se actualiza inmediatamente el texto del `QLabel` correspondiente, para que el usuario tenga confirmación visual de qué quedó cargado.

9. Arrastrar y soltar (Drag & Drop)

método `dragEnterEvent(self, event)`

```
def dragEnterEvent(self, event):
    if event.mimeData().hasUrls():
        for url in event.mimeData().urls():
            file_path = url.toLocalFile()
            if (os.path.isdir(file_path)
                or file_path.lower().endswith('.pdf')
                or file_path.lower().endswith(IMAGE_EXTENSIONS)):
                event.acceptProposedAction()
                break
```

Qt llama a este método continuamente mientras el usuario arrastra algo sobre la ventana, *antes* de soltarlo. Si al menos uno de los elementos arrastrados es una carpeta, un PDF o una imagen soportada, se acepta la acción (`acceptProposedAction()`), lo que hace que el cursor muestre que se puede soltar ahí. Si no se acepta, Windows muestra el ícono de "prohibido".

método `dropEvent(self, event)`

```
def dropEvent(self, event):
    for url in event.mimeData().urls():
        file_path = url.toLocalFile()
        if os.path.isdir(file_path):
            self.add_pdfs_from_folder(file_path)
        elif file_path.lower().endswith('.pdf'):
            self.add_pdf(file_path)
        elif file_path.lower().endswith(IMAGE_EXTENSIONS):
            self.watermark_path = file_path
            self.label_watermark.setText(...)
    self.update_pdf_label()
```

Se ejecuta cuando el usuario suelta los elementos. Recorre **todas** las URLs soltadas (se puede soltar más de un elemento a la vez) y clasifica cada una: si es una carpeta, escanea sus PDF; si es un PDF, lo agrega a la lista; si es una imagen, se convierte en la nueva marca de agua (la última imagen soltada es la que queda, ya que solo existe un `watermark_path`). Al final se refresca la etiqueta de PDFs una sola vez, para no recalcular el texto en cada iteración del bucle.

10. `processWatermark` — el orquestador del procesamiento por lotes

Este es el método más largo de la clase y actúa como director de todo el proceso cuando el usuario pulsa **Agregar Marca de Agua**. Puede dividirse en cinco etapas:

10.1 Validación

```
if not self.pdf_paths:
    QMessageBox.warning(...); return
if not self.watermark_path:
    QMessageBox.warning(...); return
```

Si falta el/los PDF o la imagen de marca de agua, se muestra un aviso y se corta la ejecución antes de tocar el disco.

10.2 Cálculo de la carpeta de salida

```
base_output = self.output_dir or os.path.join(os.path.expanduser("~"), "Downloads")
if len(self.pdf_paths) == 1:
    output_dir = base_output
else:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_dir = os.path.join(base_output, f"Recicley_Watermark_{timestamp}")
os.makedirs(output_dir, exist_ok=True)
```

`self.output_dir or ...` usa la carpeta elegida por el usuario si existe; si es cadena vacía (nunca se eligió una), cae en `Descargas`. Si solo hay un PDF, los resultados van directo a esa carpeta (comportamiento original de la app). Si hay varios, se crea una subcarpeta con marca de tiempo para no mezclar los resultados de distintas ejecuciones ni con otros archivos del usuario.

10.3 Bucle principal con barra de progreso

```
progress = QProgressDialog("Procesando archivos...", "Cancelar", 0, total, self)
...
for i, pdf_path in enumerate(self.pdf_paths):
    if progress.wasCanceled():
        break
    progress.setLabelText(f"Procesando: {filename} ({i + 1}/{total})")
    progress.setValue(i)
    QApplication.processEvents()

    out_name = self.unique_output_name(used_names, ...)
    output_path = os.path.join(output_dir, out_name)
    try:
        self.addWaterMarkToPDF(pdf_path, self.watermark_path, opacity, poppler_path, output_path)
        processed += 1
    except Exception as e:
        errors.append(f"{filename}: {e}")
```

`QProgressDialog` es una ventana modal con barra de progreso y botón "Cancelar". Como PyQt6 no refresca la interfaz automáticamente mientras un bucle de Python está corriendo, se llama manualmente a `QApplication.processEvents()` en cada iteración para que la barra y el texto se

actualicen y la ventana no se congele ("no responde"). Cada PDF se procesa dentro de un `try/except` individual: si uno falla (por ejemplo, un PDF corrupto), el error se guarda en la lista `errors` y el bucle continúa con el siguiente archivo, en lugar de abortar todo el lote.

10.4 Nombres de salida sin colisión

Antes de procesar cada archivo se calcula su nombre de salida con `unique_output_name` (sección siguiente), evitando que dos PDF de origen con el mismo nombre (por ejemplo `informe.pdf` en dos subcarpetas distintas de una misma carpeta escaneada) se sobrescriban entre sí.

10.5 Resultado final

```
if processed > 0:
    os.startfile(output_dir)
if errors:
    QMessageBox.warning(...) # resumen con éxitos y errores
elif processed > 0:
    QMessageBox.information(...) # éxito total
```

`os.startfile(output_dir)` es una llamada específica de Windows que abre la carpeta en el Explorador, para que el usuario encuentre sus archivos de inmediato. Luego se informa el resultado: si hubo errores se listan junto con los éxitos; si todo salió bien, se confirma la cantidad de archivos procesados y la ruta de salida.

11. `unique_output_name` y `addWaterMarkToPDF` — el motor que aplica la marca de agua

método estático `unique_output_name(used_names, filename)`

```
base, ext = os.path.splitext(filename)
candidate = filename
i = 1
while candidate in used_names:
    candidate = f"{base} ({i}){ext}"
    i += 1
used_names.add(candidate)
return candidate
```

Recibe el conjunto (`set`) de nombres ya usados en la corrida actual y el nombre propuesto. Si ya existe, va probando `nombre (1).pdf`, `nombre (2).pdf`, etc. hasta encontrar uno libre, lo registra en `used_names` y lo devuelve. Es un método estático porque no depende del estado de la ventana, solo de sus parámetros.

método `addWaterMarkToPDF(self, pdf_path, watermark_path, opacity, poppler_path, output_path)`

Este es el corazón técnico de la aplicación: toma **un** PDF de origen y produce **un** PDF de salida con la marca de agua aplicada en cada página. Se llama una vez por cada archivo dentro del bucle de `processWatermark`. Su lógica, por página:

- 1 **Aislar la página.** Se crea un `PdfWriter` temporal con una sola página (la actual) y se serializa a bytes en memoria (`io.BytesIO`), sin escribir nada a disco todavía.
- 2 **Renderizar a imagen.** `convert_from_bytes(...)` (de `pdf2image`, que internamente invoca Poppler usando `poppler_path`) convierte esos bytes en una imagen de mapa de bits a `RENDER_DPI` puntos por pulgada, y se fuerza el modo `RGBA` (con canal alfa) para poder componer la marca de agua encima.
- 3 **Redimensionar la marca de agua.** La imagen de marca de agua se escala exactamente al tamaño en píxeles de la página renderizada, usando el filtro de remuestreo `LANCZOS` (mejor calidad; con un *fallback* a `ANTIALIAS` por compatibilidad con versiones antiguas de Pillow).
- 4 **Aplicar la opacidad elegida.** Se extrae el canal alfa original de la marca de agua (`getchannel("A")`) y se escala proporcionalmente según el valor del slider: `a * opacity // 255`. Así, zonas ya transparentes de la imagen siguen siéndolo, y el resto se atenúa según la opacidad elegida por el usuario.
- 5 **Componer.** `Image.alpha_composite(page_image, watermark_resized)` mezcla la marca de agua sobre la página respetando la transparencia calculada en el paso anterior.
- 6 **Reempaquetar como página PDF.** La imagen compuesta se convierte a `RGB` (los PDF no usan canal alfa) y se guarda como PDF de una página en memoria, con la calidad definida en `PDF_SAVE_QUALITY` y la resolución `RENDER_DPI` (para que el tamaño físico de la página no cambie).
- 7 **Agregar al documento final.** Esa página de una hoja se vuelve a leer con `PdfReader` y se añade al `PdfWriter` acumulador que representa el documento completo de salida.

Al terminar de recorrer todas las páginas, `pdf_writer.write(...)` escribe el documento final completo en `output_path`. La lista `buffers_memoria` mantiene una referencia a cada buffer de página en memoria mientras dura el proceso, evitando que el recolector de basura de Python los libere antes de que `PdfWriter` termine de leerlos.

Por qué se convierte cada página a imagen en vez de dibujar la marca de agua directamente sobre el PDF vectorial: este enfoque garantiza que la marca de agua se vea idéntica sin importar el contenido original del PDF (texto, vectores, imágenes, formularios), a costa de "rasterizar" el documento — el PDF resultante ya no tiene texto seleccionable, es una imagen por página. Es la razón de ser de `RENDER_DPI` y `PDF_SAVE_QUALITY`: controlan el balance entre nitidez del resultado y tamaño/velocidad.

12. Punto de entrada `main()`

```
def main():
    app = QApplication(sys.argv)
    window = DragDropWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

`QApplication` es el objeto obligatorio de Qt que administra el bucle de eventos, la configuración y los recursos gráficos de toda la aplicación; solo puede existir una instancia. Se crea la ventana `DragDropWindow`, se muestra con `.show()` y `app.exec()` arranca el bucle de eventos (que procesa clics, arrastres, etc.) hasta que la ventana se cierra; su código de salida se propaga con `sys.exit()`.

13. Flujo de datos completo (paso a paso)

- 1 El usuario carga PDF(s) → `dropEvent` / `selectFileDialog` / `selectFolderDialog` → `add_pdf` / `add_pdfs_from_folder` → `self.pdf_paths` → `update_pdf_label`.
- 2 El usuario carga la imagen de marca de agua → `dropEvent` / `selectWatermarkDialog` → `self.watermark_path`.
- 3 (Opcional) El usuario elige destino → `selectOutputDialog` → `self.output_dir`.
- 4 El usuario mueve el slider → `updateOpacity` actualiza la etiqueta (el valor real se lee directo del slider al momento de procesar).
- 5 El usuario pulsa "Agregar Marca de Agua" → `processWatermark`:
 - 6 valida `pdf_paths` y `watermark_path`;
 - 7 calcula `output_dir` a partir de `output_dir` o Descargas;
 - 8 por cada PDF, calcula un nombre único (`unique_output_name`) y llama a `addWaterMarkToPDF`;
 - 9 `addWaterMarkToPDF` usa `pdf2image` + Poppler para rasterizar, `Pillow` para componer la marca de agua, y `PyPDF2` para reensamblar el PDF de salida;
 - 10 al terminar, se abre la carpeta de salida y se informa el resultado.

14. Empaquetado a `.exe` (PyInstaller)

`GUI_Watermark.spec` define cómo PyInstaller construye el ejecutable. Los puntos clave:

```

a = Analysis(
    ['GUI_Watermark.py'],
    datas=[
        ('rec.jpg', '.'),
        ('poppler-0.68.0', 'poppler-0.68.0'),
    ],
    ...
)
...
exe = EXE(..., exclude_binaries=True, name='GUI_Watermark', console=False, ...)
coll = COLLECT(exe, a.binaries, a.datas, ..., name='GUI_Watermark')

```

`datas` le indica a PyInstaller que copie `rec.jpg` y toda la carpeta `poppler-0.68.0` dentro del paquete final, para que la aplicación sea autocontenida (no depende de que esas carpetas existan manualmente junto al `.exe`: PyInstaller las coloca dentro de `_internal/`, y `get_base_path()` las localiza ahí mediante `sys._MEIPASS`). `console=False` evita que se abra una ventana de consola detrás de la interfaz gráfica. El modo de construcción es "onedir" (`COLLECT`): el resultado es una carpeta (`dist/GUI_Watermark/`) con el `.exe` y sus dependencias, que se comparte y ejecuta completa (ver `README.md` para el comando exacto de reconstrucción).

15. Solución de problemas

Síntoma	Causa probable	Solución
El <code>.exe</code> no abre / cierra inmediatamente.	Se movió o copió solo el <code>.exe</code> , sin la carpeta <code>_internal</code> .	Vuelve a descomprimir/copiar la carpeta <code>GUI_Watermark</code> completa.
Error al procesar un PDF específico dentro de un lote.	El PDF está dañado, cifrado o es un archivo no válido.	El resto del lote se procesa igualmente; revisa el mensaje de error mostrado al final para identificar el archivo y ábrelo manualmente para confirmarlo.
La marca de agua se ve muy tenue o muy fuerte.	Valor de opacidad inadecuado, o la imagen ya tiene transparencia propia.	Ajusta el slider de opacidad (0–255) antes de procesar; para más control, edita el canal alfa de la imagen origen.
El proceso tarda mucho con muchos PDFs o PDFs muy largos.	Cada página se renderiza a <code>RENDER_DPI</code> (200 DPI) de forma individual; es un proceso intensivo en CPU.	Es esperable en lotes grandes; se puede cancelar en cualquier momento con el botón "Cancelar" de la barra de progreso.